LABORATORIO MULTIMEDIA · ANDROID STUDIO

La forja del servidor central

Consola de monitorización, ExoPlayer y señal de alarma local

Media3 · ExoPlayer · Kotlin · activity_laboratorio3.xml



ARQUITECTURA DE LA CONSOLA

Una consola para sintonizar y vigilar canales





El operador selecciona un puesto remoto, observa su vídeo y consulta el estado de conexión. Al terminar la señal, una alarma sonora local se reproduce de forma independiente.

`ExoPlayer (streaming) + MediaPlayer (audio local)`

DISEÑO DE INTERFAZ · XML

La pantalla reúne selector, vídeo y estado

En activity_laboratorio3.xml: **Spinner** arriba para puestos remotos, **PlayerView** en el centro para vídeo y **TextView** abajo para diagnóstico y estado.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent" android:layout_height="match_parent"
  android:orientation="vertical" android:padding="16dp">
  <Spinner android:id="@+id/spCanales" android:layout_width="match_parent"
    android:layout_height="wrap_content" />
  <androidx.media3.ui.PlayerView android:id="@+id/playerView"
    android:layout_width="match_parent" android:layout_height="0dp"
    android:layout_weight="1" />
  <TextView android:id="@+id/tvEstado" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:text="@string/estado_inicial" />
</LinearLayout>
```

DEPENDENCIAS Y CONFIGURACIÓN

Preparar Media3 y el acceso a Internet

build.gradle.kts (Module :app)

```
dependencies {
    val media3Version = "1.5.0" // Mantener alineadas
    implementation("androidx.media3:media3-exoplayer:$media3Version")
    implementation("androidx.media3:media3-ui:$media3Version")
    implementation("androidx.media3:media3-common:$media3Version")
}
```

Verifique sincronizar la misma versión para todos los módulos de Media3.

AndroidManifest.xml

```
<manifest
xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Requerido para flujos RTSP, HLS y HTTP -->
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:usesCleartextTraffic="true"
        ... >
    </application>
</manifest>
```

cleartextTraffic es necesario si el servidor de pruebas emite sobre HTTP plano.

CATÁLOGO DE FUENTES

Un mapa vincula cada puesto con su URL

Un mapa inmutable asocia cada puesto con su flujo; sus claves alimentan el Spinner mediante un ArrayAdapter.

Mapa de canales

```
private val canales = mapOf(
    "Minas Tirith" to "PENDIENTE_CATEDRA",
    "Isengard" to "PENDIENTE_CATEDRA",
    "Rivendell" to "PENDIENTE_CATEDRA"
)
```

Spinner en onCreate()

```
val nombresPuestos = canales.keys.toList()
val adapter = ArrayAdapter(
    this,
android.R.layout.simple_spinner_item,
    nombresPuestos
).apply {
    setDropDownViewResource(

android.R.layout.simple_spinner_dropdown_item)
}
binding.spCanales.adapter = adapter
```

Las URL son marcadores de posición, no direcciones reales. Sustitúyelas por las facilitadas por la cátedra.

CONTROL DE REPRODUCCIÓN

Cambiar de canal actualiza ExoPlayer

El listener obtiene la URL del puesto, detiene la alarma previa y asigna el MediaItem; después prepara e inicia la reproducción.

```
binding.playerView.player = player
binding.spCanales.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
    override fun onItemSelected(parent: AdapterView<*>, v: View?, pos: Int, id: Long) {
        val puesto = parent.getItemAtPosition(pos).toString()
        val url = canales[puesto] ?: return
        detenerAlarma()
        player?.apply {
            setMediaItem(MediaItem.fromUri(url))
            prepare()
            playWhenReady = true
        }
    }
    override fun onNothingSelected(parent: AdapterView<*>) {}
}
```

Listener del Spinner enlazando la interfaz con el ciclo del reproductor.

MONITORIZACIÓN Y TELEMETRÍA

El estado del reproductor informa al operador

STATE_BUFFERING

«Cargando Búfer...»

Carga inicial o búfer por congestión de red.

STATE_READY

«Transmitiendo en vivo»

Listo o en pausa; no implica directo.

STATE_ENDED

«Conexión terminada»

Fin normal de reproducción de la pista multimedia.

```
player?.addListener(object : Player.Listener {
    override fun onPlaybackStateChanged(playbackState: Int) {
        when (playbackState) {
            Player.STATE_BUFFERING -> binding.tvEstado.text = "Cargando Búfer..."
            Player.STATE_READY     -> binding.tvEstado.text = "Transmitiendo en vivo"
            Player.STATE_ENDED     -> { binding.tvEstado.text = "Conexión terminada"
                dispararAlarma() }
            Player.STATE_IDLE      -> binding.tvEstado.text = "En espera"
        }
    }
})
```

SEÑALIZACIÓN ACÚSTICA

La alarma local avisa del fin de señal

Guarda campanas_alarma.mp3 en res/raw. Al llegar a STATE_ENDED, llama a dispararAlarma() para crear un MediaPlayer y ejecutar start().

Reproducción independiente: Android se encarga del audio; no hace falta crear un Thread o Coroutine manualmente.

Libera la instancia anterior o al completar la alarma para evitar fugas y bloqueos de recursos de audio.

```
private var mediaPlayerAlarma: MediaPlayer? = null

private fun dispararAlarma() {
    mediaPlayerAlarma?.release()
    mediaPlayerAlarma = MediaPlayer.create(
        this, R.raw.campanas_alarma
    ).apply {
        setOnCompletionListener {
            it.release(); mediaPlayerAlarma = null
        }
        start()
    }
}
```

Gestión del ciclo de vida y finalización del MediaPlayer local.

RESILIENCIA Y ERRORES

Una interrupción de red no es un final normal

Cuando se corta la conexión o el servidor cae, ExoPlayer emite onPlayerError, no STATE_ENDED.

- **STATE_ENDED:** Señal finalizada por diseño de pista.
- **PlaybackException:** Timeout, error DNS o pérdida de red.

Gestiona el fallo por separado y evita alarmas duplicadas con una marca de alarma activa.

```
import androidx.media3.common.PlaybackException.*

override fun onPlayerError(error: PlaybackException) {
    val causa = when (error.errorCode) {
        ERROR_CODE_IO_NETWORK_CONNECTION_FAILED,
        ERROR_CODE_IO_NETWORK_CONNECTION_TIMEOUT ->
            "Error de red: servidor inaccesible"
        else -> "Fallo en reproducción
(${error.errorCodeName})"
    }
    binding.tvEstado.text = causa
    if (!alarmaActiva) dispararAlarma()
}
```

Captura de PlaybackException diferenciando errores de red de cierres de pista. Mantén alarmaActiva sincronizada con la alarma.

CICLO DE VIDA DE LA ACTIVIDAD

El ciclo de vida mantiene el reproductor bajo control

01	02	03	04	05
Crear	Vincular	Cargar	Ejecutar	Liberar
ExoPlayer.Builder	playerView.player	setMediaItem()	prepare() / play()	player.release()

onStart() / onResume()

```
private fun inicializarPlayer() {
    player =
ExoPlayer.Builder(this).build().also { exo ->
        binding.playerView.player = exo
    }
}
```

onStop() / onDestroy()

```
override fun onStop() {
    super.onStop()
    detenerAlarma()
    player?.release()
    player = null
}
```